

My Exam

Task 1:

Task 2: Gaussian Processes Regression

Part 2.1

Port 2

2.1 determine likelihood for the regression model in $y_n = f(x_n) + \epsilon_n$

- we have a following model: $y_n = f(x_n) + \epsilon_n$, where $\epsilon_n \sim \mathcal{N}(0, \sigma^2)$
- likelihood for a single-point: $p(y_n | f_n) = \mathcal{N}(y_n | f(x_n), \sigma^2)$
- likelihood for entire dataset: $p(y | f) = \mathcal{N}(y | f, \sigma^2 I)$

$p(y | f) = \mathcal{N}(y | f, \sigma^2 I_n)$

Given the values $f = [f(x_1), \dots, f(x_n)]$, the vector of observed outputs y follows a multivariate norm with $\mu = f$ and $\Sigma = \sigma^2 I_n$

- $p(y | f)$ - likelihood: the probability of observing the output y given the function values f .
- $\mathcal{N}(y | f, \sigma^2 I_n)$ - multivariate Gaussian PDF evaluated at y .
- the reason why we can write our obs as $y \sim \mathcal{N}(f, \sigma^2 I_n)$ because our obs are $\perp$

Figure 1: My Caption

Part 2.2

In hand solution

22 Determine the analytical prior prediction distribution $p(y^* | x^*=1)$ for $y^* = y(x^*)$

- $f^* := f(x^*) \rightarrow$ function value at test point
- $y^* := f^* + \epsilon^* \rightarrow$ noisy observation at test pt.
- from the GP Prior $f^* \sim \mathcal{N}(0, k(x^*, x^*))$
- from the noise model $\epsilon^* \sim \mathcal{N}(0, \sigma^2)$, independent of f^*

$\therefore y^* = f^* + \epsilon^* \sim \mathcal{N}(0, k(x^*, x^*) + \sigma^2)$

$k = 0.7$
 $\ell = \frac{1}{2}\sqrt{2}$
 $\sigma = \frac{1}{5} = 0.2$

$$p(y^* | x^*=1) = \mathcal{N}(0, k(x^*, x^*)) = \mathcal{N}(0, k(1, 1))$$

$$\Rightarrow k(x, x') = k^2 \exp\left(-\frac{1}{2\ell^2} \|x - x'\|_2^2\right)$$

$$k(1, 1) = 0.7^2 \exp\left(-\frac{1}{2 \cdot \left(\frac{\sqrt{2}}{2}\right)^2} \|1 - 1\|_2^2\right)$$

$$= 0.49 \exp\left(-\frac{1}{2 \cdot \frac{2}{4}} \|1 - 1\|_2^2\right)$$

$$f^*(x=1) = 0.49 \exp\left(-\cancel{\|1 - 1\|_2^2}\right)$$

to obtain $y^*(x=1) = f^* + \sigma^2$

$$= k^2 + \sigma^2$$

$$= 0.7^2 + 0.2^2 = 0.53$$

$\therefore p(y^* | x=1) = \mathcal{N}(0, k^2 + \sigma^2) = \underline{\underline{\mathcal{N}(0, 0.53)}}$

Figure 2: My Caption

My solution

```
1 x = jnp.array([-2.17, 1.99, 0.57, -3.01, -1.16, 3.30, -4.85, -0.86]).reshape(-1, 1) # need to
   ↪ reshape to (n, 1)
```

```

2 print(f"Shape of X: {x.shape}")
3 y = jnp.array([0.88 ,0.46 ,-0.06, 0.98, 0.45,0.88 ,-0.66, 0.05])[:, None]
4 print(f"Shape of y: {y.shape}")
5
6 # hyperparameters
7 kappa = 0.7
8 ell = 0.5 * jnp.sqrt(2)
9 sigma = 1/5
10
11 # Define the kernel
12 kernel = GP.StationaryIsotropicKernel(kernel_fun=GP.squared_exponential, kappa=kappa, lengthscale=
    ↪ ell)
13
14 # define the model
15 GP_ = GP.GaussianProcessRegression(X=x, y=y, kernel=kernel, kappa=kappa, lengthscale=ell, sigma=
    ↪ sigma)
16
17 # prior predictive distribuion
18
19 # x* = 1 # the prediction point
20 x_star = jnp.array([1])
21
22 # get the mean and variance from the prior_predictive of y star
23 prior_pred_y_mean, prior_pred_y_var = GP_.prior_predictive_y_star(Xstar=x_star)
24
25 # we can get the same by using the
26 prior_pred_f_mean, prior_pred_f_mean = GP_.prior_predictive_f_star(Xstar=x_star)
27
28 print(f"Prior pred mean of y*: {prior_pred_y_mean.item():3.2f}")
29 print(f"Prior pred var of y*: {prior_pred_y_var.item():3.2f}")
30
31 print(f"Prior pred mean of f*: {prior_pred_f_mean.item():3.2f}")
32 print(f"Prior pred var of f*: {prior_pred_f_mean.item():3.2f}")

```

Teachers solution

Question 2.2

The prior distribution of $f^* = f(x^*)$ for any x^* is given by $p(f^*) = \mathcal{N}(f^*|0, k_1(x^*, x^*)) = \mathcal{N}(f^*|0, \kappa^2)$.

Hence, the prior predictive distribution is given by

$$p(y^*|x^* = 1) = \mathcal{N}(y^*|0, \kappa^2 + \sigma^2) = \mathcal{N}(y^*|0, 0.53)$$

```

import numpy as np
kappa = 0.7
ell = 0.5*np.sqrt(2)
sigma = 0.2

print(f'p(y^*|x^*=1) = N(y^*|0, {kappa**2 + sigma**2:3.2f})')
p(y^*|x^*=1) = N(y^*|0, 0.53)

```

Figure 3: Teachers solution

```

1 print(f"p(y^* | x^*=1) = N(y^*| 0, {kappa**2 + sigma**2:3.2f})")

```

Part 2.3

2.3 Determine the analytical post. pred. dist $p(y^*|y, x^*=227)$ for $y^* = y(x^*)$

- we start w/ prior on function values, $f \sim GP \sim (0, k(x, x'))$, this means any finite collection of function values follow mult. Gauss.
- we add noisy observations, each observation y_n is: $y_n = f_n + \epsilon_n$, $\epsilon_n \sim N(0, \sigma^2)$, so the observed y vector is "noisy" version of f .
- posterior over test value $f^* = f(x^*)$: given observed y and the GP prior, the posterior over f^* is:

$$p(f^*|y, x^*) = N(\mu_*, \sigma_*^2)$$

$$\mu_* = k_*^T (K + \sigma^2 I)^{-1} y$$

$$\sigma_*^2 = k(x^*, x^*) - k_*^T (K + \sigma^2 I)^{-1} k_*$$

μ_* : $N \times 1$ vector of mean
 σ_*^2 : $N \times N$ matrix of train inputs
 k_* : $N \times 1$ vector between test input x^* and training input x .

- add noise to get dist. over y^*
 $\rightarrow$ we want $y^* = f^* + \epsilon$, $\epsilon \sim N(0, \sigma^2) \Rightarrow p(y^*|y, x^*) = N(\mu_*, \sigma_*^2 + \sigma^2)$

Part 2.4

2.4 Determine the analytical expression for the marginal likelihood $p(y|h, l, \sigma)$ and compute the num. val.

$\log p(y|h, l, \sigma)$

- to find the marginal likelihood of the data $p(y)$, we need to marginalize f out of $N(f, \sigma^2 I, l, \sigma)$

$$p(y) = \int p(y|f) p(f) df$$

$$p(y) = \int \underbrace{p(y|f, \sigma^2 I)}_{\text{likelihood}} \underbrace{p(f|h, l)}_{\text{prior}} df$$

$$p(y) = N(y|0, K + \sigma^2 I) = -\frac{1}{2} y^T (K + \sigma^2 I)^{-1} y - \frac{1}{2} \log |K + \sigma^2 I| - \frac{n}{2} \log(2\pi)$$

(this is just standard multivariate normal dist.)
 Python: `np.linalg.pfE(y, mean=0, cov=K + 515*sigma**2, log_det=N)`

- using the function `log-marginal-likelihood` we get the value $\log p(y) = -6.65$

Figure 4: Marinal likelihood

```
1 log_marginal = GP_.log_marginal_likelihood(kappa=kappa, lengthscale=ell, sigma=sigma)
2
3 print(f"The value of log marginal marginal likelihood log p(y) = {log_marginal:.2f} ")
```

Teachers solution

Part 2.5

```
1 x_star = jnp.array([1])
2
3 mu_f, var_f = GP_.predict_f(Xstar=x_star)
4 print(f"The analytical posterior distribution p(f*|y, x*=1) = N(f*|{mu_f.item():.2f}, {var_f.item()
   ↪ :.2f})")
5 mu_y, var_y = GP_.predict_y(Xstar=x_star)
6 print(f"The analytical posterior distribution p(y*|y, x*=1) = N(f*|{mu_y.item():.2f}, {var_y.item()
   ↪ :.2f})")
```

Question 2.4

Gaussian process regression with a Gaussian likelihood is a conjugate model and therefore, we can compute the marginal likelihood analytical, which is given by:

$$p(\mathbf{y}|\kappa, \ell, \sigma) = \mathcal{N}(\mathbf{y}|\mathbf{0}, \mathbf{K} + \sigma^2\mathbf{I}), \quad (1)$$

where $\mathbf{K}$ is the covariance matrix for the latent function values $\mathbf{f}$, i.e. $\text{cov}(\mathbf{f}, \mathbf{f}) = \mathbf{K}$ and σ is the standard deviation of the noise.

We can evaluate the log PDF of the distribution above directly, or we can use the code from the course (week5):

```
from week5 import squared_exponential
from week5 import StationaryIsotropicKernel
from week5 import GaussianProcessRegression

# data
x = np.array([-2.17, 1.99, 0.57, -3.01, -1.16, 3.30, -4.85, -0.86])[:, None]
y = np.array([0.88, 0.46, -0.06, 0.98, 0.45, 0.88, -0.66, 0.05])[:, None]

# using the code from the course
gp = GaussianProcessRegression(x, y, StationaryIsotropicKernel(squared_exponential))
print(f'log marginal likelihood: {gp.log_marginal_likelihood(kappa, ell, sigma):3.2f} (course code)')

# or by evaluating a multivariate normal directly:
from scipy.stats import multivariate_normal as mvn
kern = StationaryIsotropicKernel(squared_exponential)
C = kern.construct_kernel(x, x, kappa=kappa, lengthscale=ell) + sigma**2*np.identity(8)
print(f'log marginal likelihood: {mvn.logpdf(y.ravel(), np.zeros(8), C):3.2f} (direct evaluation)')

log marginal likelihood: -6.65 (course code)
log marginal likelihood: -6.65 (direct evaluation)

Therefore,
```

$$\log p(\mathbf{y}|\kappa, \ell, \sigma) = \log \mathcal{N}(\mathbf{y}|\mathbf{0}, \mathbf{K} + \sigma^2\mathbf{I}) \approx -6.65.$$

Figure 5: alt text

Part 2.6

Part 2.7

```
1 def squared_exponential_2(x1, x2, c1, c2, lengthscale):
2     return ((1 + jnp.abs(x1 - x2) / (2 * lengthscale**2))**(-1)) + x1 * x2
3
4 x_star = jnp.array([-1]).reshape(-1)
5 kernel2 = squared_exponential_2(x1=x, x2=x_star, c1=1, c2=1, lengthscale=jnp.sqrt(1/2))
6
7 # Print the kernel values with 2 decimal places
8 print(jnp.round(kernel2, 2))
```

Teachers solution

```
1 import numpy as np
2
3 # Define data
4 x = np.array([-2.17, 1.99, 0.57, -3.01, -1.16, 3.30, -4.85, -0.86])[:, None]
5
6
7
8 k2 = lambda x, xprime: 1./(1 + np.abs(x - xprime)) + x * xprime
9
10 xstar = np.array(-1)
11 for xn in x:
12     print(f'{np.array2string(k2(xn, xstar), precision=2)}', end=', ')
```

Task 3: XX

Task 4: XX

Task 5: XX